

making sure you install the correct GSM modem driver on the PC. An example of such a set-up is shown in Figure 1. Once the set-up is done, you are ready to execute the AT commands.

AT commands for SMS exchange: AT commands include the prefix *AT* or *at* which indicates the beginning of the command and are terminated by a carriage return *<CR>*. The syntax of AT commands is as follows:

AT <command name><CR>

Multiple AT commands can be sent in a single command line by separating them with a semicolon. In extended AT commands, the command name is prefixed by '+'. AT commands are usually followed by responses that start as well as end with carriage returns and a line feed, e.g.:

<CR><LF><response><CR><LF>

Some of the important commands commonly used in SMS software are mentioned in Table 1.

Sending SMSs from a hyper terminal: Before implementing the commands in the software, it is always preferable to use the hyper terminal to send the command to the modem and verify the response. To send the command from a hyper terminal, open the hyper terminal and select the COM port to which the GSM device is connected. Set the baud rate at 9600 bps, data bits as 8, parity as 'none' and 1 stop bit. After configuration, the hyper terminal can be used for sending AT commands to the device. While executing commands on the hyper terminal, use the *Enter* key for <CR>. The screenshot given in Figure 2 depicts the command used for checking the device, defining the message format, setting the message centre number, and then sending the SMS from the hyper terminal.

SMS software: The successful transfer of the SMS from the hyper terminal confirms that the set-up is correct and the device is responding well to AT commands. This is the right time to start developing SMS software that controls your mobile phone, for which you can use the Linux or Windows platforms, the Eclipse/Netbeans IDE, the MySQL database and Java programming language. To access the serial port from the software, the serial port library is an essential prerequisite. For easier access to software capabilities, a friendly graphical user interface has to be designed. The database is required to store the messages extracted from the device using software and the messages sent to the network. Here is a Java-based example for those interested in developing software and integrating the SMS feature into it.

To check the connection:

```
String command1 = "AT\r\n";
```

To check the SIM:

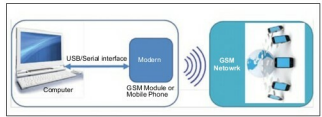


Figure 1: PC to GSM communication set-up

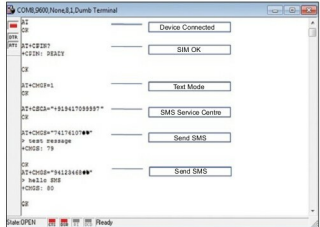


Figure 2: AT commands executed from the hyper terminal

```
String command2 ="AT+ CPIN?\r\n";
```

Text mode:

```
String command3 ="AT+CHGF=1\r\n";
```

SMS service centre:

```
String command4 = "AT+CSCA=\"\" + address + "\"\r\n";
```

To send an SMS:

```
String command5 ="AT+CHGS=\"\" + mobile + "\"\r\n";
```

and

```
String command6 =message + '\032';
```

The keywords, address, mobile and message used here are Java strings for the SMS service centre address, mobile number and SMS text message, respectively (e.g., String address = "+91941xxxx997").

Extending the software to create an SMS gateway: The SMS software can also be extended as an SMS gateway. A gateway can receive the messages from the other machines attached to the wire-line (e.g., wired LAN) or wireless network (Wi-Fi), and can forward the messages to the GSM network. A socket based client-server approach can be used